that employ *SA_INTERRUPT* will have all local interrupts disabled! Hence, you can see that reducing the time allocated for this (that is, the time for which the interrupts remain disabled) is vital when it comes to the overall performance of the system.

The logical part is very vital when you decide when you want to draw the line that separates the top and bottom half (terminologically it should be *part* instead of half). The whole point of this division is to improvise the system performance. You can see that by this division we can actually postpone many works. You can perform them when the system is 'less busy'. In most cases, these bottom halves run just after the interrupt returns. This separation will enable us to have the best system performance.

There are various mechanisms that allow you to implement the bottom part effectively. Historically, Linux offered the 'bottom half' (BH) for meddling with all the bottom parts. The original interface was quite simple and somewhat elegant, providing a statically created list of 32 bottom halves. The top part set the bottom to run by assigning a bit in a 32-bit integer. Each BH was globally synchronised and no two BHs were allowed to run in parallel.

But this model had many disadvantages, the most important being the inflexibility associated with it. The queue had a linked list of functions, which could be called, and these would be executed during the process as per the schedule. And the driver could register the bottom halves in respective queues. But this couldn't replace the old BH entirely. Unfortunately, we were unable to handle sub-systems like networking. But during 2.3 kernel development series, programmers addressed this problem by introducing *softirqs* and *tasklets*. The following code illustrates *softirq* 'linked portion' of the interrupt (header) file in kernel source:

```
#define set_softirq_pending(x) (local_softirq_pending() = (x))
#define or_softirq_pending(x)  (local_softirq_pending() |= (x))
#endif

/* Some architectures might implement lazy enabling/disabling of
 * interrupts. In some cases, such as stop_machine, we might want
 * to ensure that after a local_irq_disable(), interrupts have
 * really been disabled in hardware. Such architectures need to
 * implement the following hook.
 */
#ifndef hard_irq_disable
#define hard_irq_disable() do { } while(0)
#endif

/* PLEASE, avoid to allocate new softirqs, if you need not _really_ high
   frequency threaded job scheduling. For almost all the purposes
   tasklets are more than enough. F.e. all serial device BHs et
   al. should be converted to tasklets, not to softirqs.
 */

enum
{
        HI_SOFTIRQ=0,
        TIMER_SOFTIRQ,
        NET_TX_SOFTIRQ,
        NET_RX_SOFTIRQ,
        BLOCK_SOFTIRQ,
        TASKLET_SOFTIRQ,
        SCHED_SOFTIRQ,
#ifdef CONFIG_HIGH_RES_TIMERS
        HRTIMER_SOFTIRQ,
#endif
        RCU_SOFTIRQ,    /* Preferable RCU should always be the last softirq
*/

        NR_SOFTIRQS
};

/* softirq mask and active fields moved to irq_cpustat_t in
 * asm/hardirq.h to get better cache usage.  KAO
 */

struct softirq_action
{
        void (*action)(struct softirq_action *);
};

asmlinkage void do_softirq(void);
asmlinkage void __do_softirq(void);
extern void open_softirq(int nr, void (*action)(struct softirq_action *));
extern void softirq_init(void);
#define __raise_softirq_irqoff(nr) do { or_softirq_pending(1UL << (nr)); }
while (0)
extern void raise_softirq_irqoff(unsigned int nr);
extern void raise_softirq(unsigned int nr);

/* This is the worklist that queues up per-cpu softirq work.
 *
 * send_remote_sendirq() adds work to these lists, and
 * the softirq handler itself dequeues from them.  The queues
 * are protected by disabling local cpu interrupts and they must
 * only be accessed by the local cpu that they are for.
 */
DECLARE_PER_CPU(struct list_head [NR_SOFTIRQS], softirq_work_list);

/* Try to send a softirq to a remote cpu.  If this cannot be done, the
 * work will be queued to the local cpu.
 */
extern void send_remote_softirq(struct call_single_data *cp, int cpu, int
softirq);

/* Like send_remote_softirq(), but the caller must disable local cpu
interrupts
 * and compute the current cpu, passed in as 'this_cpu'.
```